Case Study 3: Retail Billing with Multiple Discount Rules

Activity sheet case study #7 | Concepts: Tuples, tuple unpacking, if-elif chains, loops, lists, strings, operators

1. Problem Statement

A shop sells items from different categories. Develop a program that processes several products with quantities, applies category-specific tax, gives a threshold-based discount subject to a maximum concession limit and prints an itemised invoice.

2. Objectives

- Process a cart stored as a list of tuples using loops and tuple unpacking.
- Select tax rates with if-elif chains based on category strings.
- Apply tiered discount rules with an upper cap.
- Produce a neatly formatted itemised invoice and reject invalid lines.

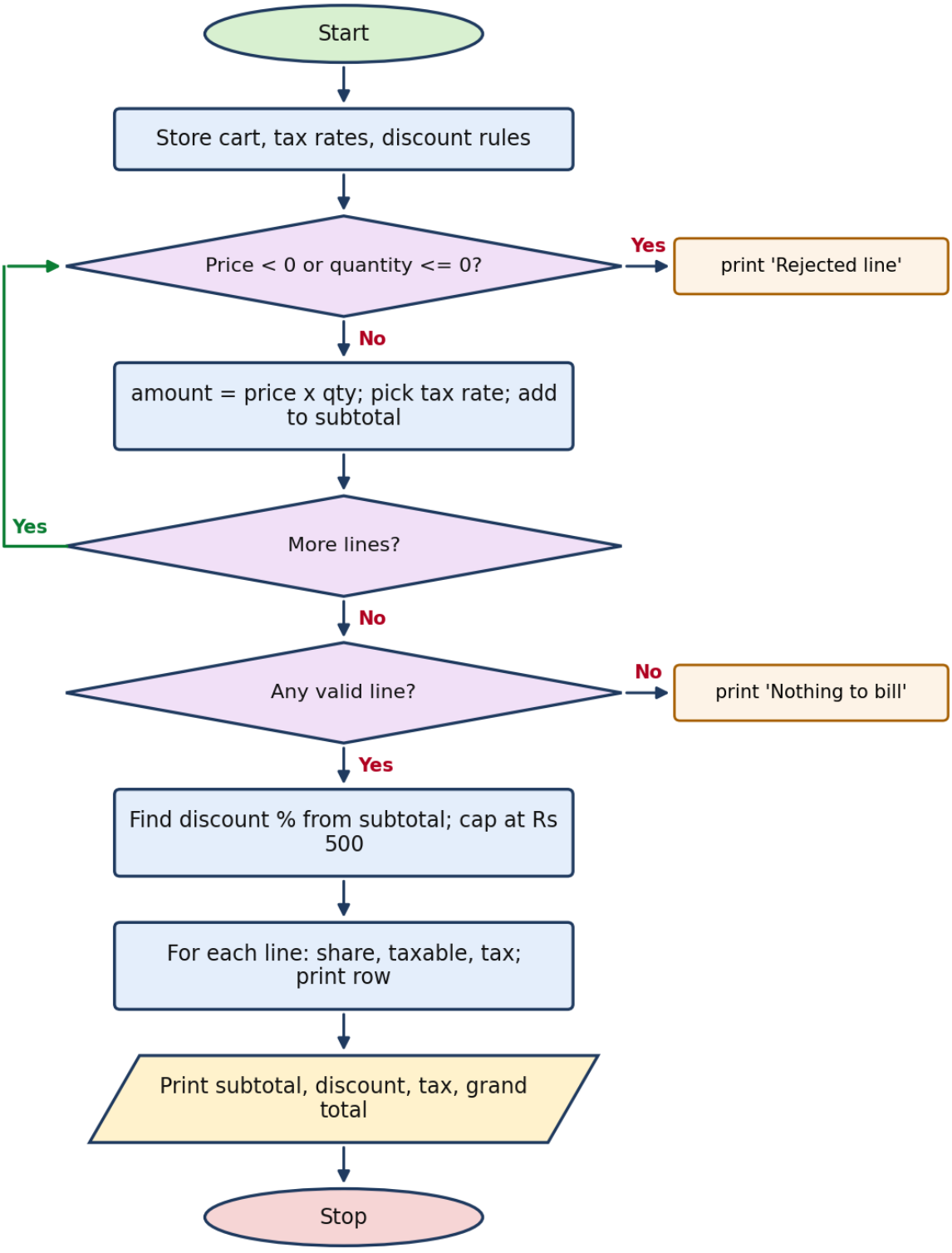
3. Assumptions

- Each cart line is (item name, category, unit price, quantity).
- Tax rates: Grocery 5 %, Electronics 18 %, Clothing 12 %, any other category 10 %.
- Discount on the pre-tax subtotal: 5 % if subtotal >= Rs 2000, 10 % if subtotal >= Rs 5000; the discount never exceeds Rs 500.
- The discount is shared among items in proportion to their amounts, and tax is charged on the discounted amount.
- A line with a negative price or a quantity of zero or less is rejected.

4. Algorithm

1. Store the cart and the tax/discount constants.
2. For each line, unpack it and reject it if price < 0 or quantity <= 0.
3. Otherwise compute amount = price x quantity, choose the tax rate from the category, store the line in a valid list and add to the subtotal.
4. If the valid list is empty, print 'Nothing to bill' and stop.
5. Decide the discount percentage from the subtotal and compute the discount; if it exceeds Rs 500, set it to 500.
6. For every valid line: share = amount / subtotal x discount; taxable = amount - share; tax = taxable x rate / 100; accumulate tax and print the invoice row.
7. Grand total = subtotal - discount + total tax.
8. Print subtotal, discount, tax and grand total.

5. Flowchart



6. Pseudocode

```
START
  SET tax rates and discount rules ; subtotal <- 0 ; valid <- empty list
  FOR each (name, category, price, qty) in cart
      IF price < 0 OR qty <= 0 THEN PRINT "Rejected"; CONTINUE
      amount <- price * qty
      IF category = "Grocery" THEN rate <- 5
      ELSE IF category = "Electronics" THEN rate <- 18
```

```
        ELSE IF category = "Clothing" THEN rate <- 12
        ELSE rate <- 10
        APPEND (name, category, price, qty, amount, rate) to valid
        subtotal <- subtotal + amount
    END FOR
    IF valid is empty THEN PRINT "Nothing to bill" ; STOP
    IF subtotal >= 5000 THEN pct <- 10
    ELSE IF subtotal >= 2000 THEN pct <- 5 ELSE pct <- 0
    discount <- subtotal * pct / 100
    IF discount > 500 THEN discount <- 500 (capped)
    FOR each line in valid
        share <- amount / subtotal * discount
        tax <- (amount - share) * rate / 100
        total_tax <- total_tax + tax ; PRINT row
    grand <- subtotal - discount + total_tax
    PRINT subtotal, discount, total_tax, grand
STOP
```

7. Python Source Code

```
# Case Study 7: Retail Billing with Multiple Discount Rules
# ---- INPUT DATA ----
# (item name, category, unit price, quantity)
cart = [
    ("Rice 5kg",   "Grocery",     420.00, 2),
    ("Headphones", "Electronics", 1999.00, 1),
    ("T-Shirt",    "Clothing",    499.00, 3),
    ("Notebook",   "Stationery",  60.00, 10),
]
# ---- END DATA ----

TAX_GROCERY = 5
TAX_ELECTRONICS = 18
TAX_CLOTHING = 12
TAX_OTHER = 10
DISCOUNT_LIMIT_1 = 2000
DISCOUNT_PERCENT_1 = 5
DISCOUNT_LIMIT_2 = 5000
DISCOUNT_PERCENT_2 = 10
MAX_DISCOUNT = 500

valid = []
subtotal = 0.0
for line in cart:
    name, category, price, qty = line
    if price < 0 or qty <= 0:
        print("Rejected line:", name, "(invalid price or quantity)")
        continue
    amount = price * qty
    if category == "Grocery":
        rate = TAX_GROCERY
    elif category == "Electronics":
        rate = TAX_ELECTRONICS
    elif category == "Clothing":
        rate = TAX_CLOTHING
    else:
        rate = TAX_OTHER
    valid.append((name, category, price, qty, amount, rate))
    subtotal = subtotal + amount

if len(valid) == 0:
    print("Nothing to bill.")
else:
    discount_percent = 0
    if subtotal >= DISCOUNT_LIMIT_2:
        discount_percent = DISCOUNT_PERCENT_2
    elif subtotal >= DISCOUNT_LIMIT_1:
        discount_percent = DISCOUNT_PERCENT_1
    discount = subtotal * discount_percent / 100
    capped = False
    if discount > MAX_DISCOUNT:
        discount = MAX_DISCOUNT
        capped = True

    print("=" * 74)
    print("{:^74}".format("ITEMISED INVOICE"))
    print("=" * 74)
    print("{:<12}{:<13}{:>4}{:>9}{:>10}{:>9}{:>6}{:>10}".format(
          "Item", "Category", "Qty", "Rate", "Amount", "Disc.", "Tax%", "Tax"))
    print("-" * 74)
    total_tax = 0.0
    for line in valid:
        name, category, price, qty, amount, rate = line
        share = amount / subtotal * discount
        tax = (amount - share) * rate / 100
        total_tax = total_tax + tax
        print("{:<12}{:<13}{:>4}{:>9.2f}{:>10.2f}{:>9.2f}{:>6}{:>10.2f}".format(
              name, category, qty, price, amount, share, rate, tax))
    print("-" * 74)

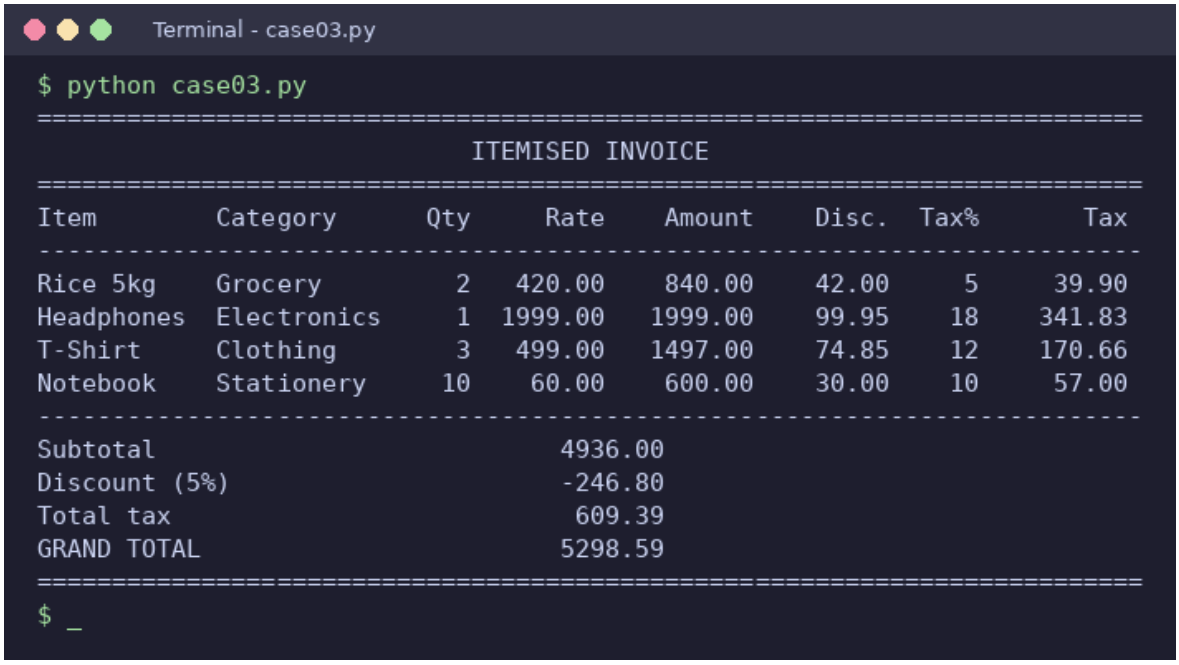
    grand_total = subtotal - discount + total_tax
    discount_label = "Discount (" + str(discount_percent) + "%)"
    if capped:
        discount_label = discount_label + " capped"
    print("{:<30}{:>12.2f}".format("Subtotal", subtotal))
    print("{:<30}{:>12.2f}".format(discount_label, -discount))
    print("{:<30}{:>12.2f}".format("Total tax", total_tax))
    print("{:<30}{:>12.2f}".format("GRAND TOTAL", grand_total))
    print("=" * 74)
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default four-item cart (5 % tier)	4 items, subtotal 4936	5 % discount applied	`Discount (5%) -246.80 | GRAND TOTAL 5298.59`	**PASS**
2	Cap on maximum concession	1 laptop at Rs 60000	10 % = 6000 reduced to 500; total 70210.00	`Discount (10%) capped -500.00 | GRAND TOTAL 70210.00`	**PASS**
3	Below discount threshold	1 pen set, subtotal 50	No discount; 50 + 5 tax = 55.00	`Discount (0%) -0.00 | GRAND TOTAL 55.00`	**PASS**
4	Exactly on the Rs 2000 boundary	subtotal exactly 2000	Boundary qualifies for 5 %	`Discount (5%) -100.00`	**PASS**
5	Invalid quantity	quantity 0	Line rejected; no invoice	`Rejected line: Bad (invalid price or quantity) | Nothing to bill.`	**PASS**
6	Mixed valid and invalid lines	one good, one negative price	Only Milk billed: 100 + 5 tax = 105.00	`Rejected line: Oops (invalid price or quantity) | GRAND TOTAL 105.00`	**PASS**

9. Output Screenshot

Terminal output of `python case03.py` using the default data:



10. Conclusion

The invoice generator combines tuples, loops, an if-elif tax table and a capped tiered discount. Distributing the discount in proportion to item amounts keeps tax correct per category, and the tests at the 2000 boundary and the Rs 500 cap confirm the rules.